

```

=== Average fare by class ===
Pclass
1      84.15
2      20.66
3       13.68
Name: Fare, dtype: float64

=== Correlation Matrix ===
      Survived  Pclass   Age   Fare  FamilySize
Survived    1.000  -0.338 -0.065  0.257    0.017
Pclass      -0.338   1.000 -0.340 -0.549    0.066
Age         -0.065  -0.340  1.000  0.097   -0.246
Fare         0.257  -0.549  0.097  1.000    0.217
FamilySize   0.017   0.066 -0.246  0.217    1.000

```

Double-click (or enter) to edit

✓ ADVANCE DATA VISUALIZATION WITH MATPLOTLIB AND SEABORN

Survival count bar chart.

```

import pandas as pd
import matplotlib.pyplot as plt
import os

# Check if the file exists, if not, download it
file_name = 'Titanic-Dataset.csv'
if not os.path.exists(file_name):
    print(f"'{file_name}' not found. Attempting to download from a public source.")
    # Assuming a common public source for the titanic dataset
    # This URL provides 'titanic.csv', so we'll rename it during download
    !wget -q https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv -O {file_name}
    if os.path.exists(file_name):
        print(f"'{file_name}' downloaded successfully.")
    else:
        raise FileNotFoundError(f"Could not find or download '{file_name}'. Please ensure the file is in the environment or provide a valid URL.")

# Data loading
df = pd.read_csv(file_name)

# Missing value handling
df.drop(columns = ['Cabin'], inplace=True, errors='ignore')
age_medium = df['Age'].median()
df['Age'].fillna(age_medium, inplace=True)
embarked_mode = df['Embarked'].mode()[0]
df['Embarked'].fillna(embarked_mode, inplace=True);

# Feature Engineering
bins = [0, 12, 18, 35, 60, 100]
labels = ['Child', 'Teenager', 'Young Adult', 'Adult', 'Senior']
df['Age_Group'] = pd.cut(df['Age'], bins=bins, labels=labels)
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1
df['IsAlone'] = (df['FamilySize'] == 1).astype(int)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

survival_counts = df['Survived'].value_counts()
colors = ['#FF6B6B', '#4ECDC4'] # red=died, teal=survived
axes[0].bar(['Did Not Survive (0)', 'Survived (1)'],
            survival_counts.values, color=colors, width=0.5, edgecolor='black')
axes[0].set_title('Survival Count', fontsize=14, fontweight='bold')
axes[0].set_ylabel('Number of Passengers')
# Add value labels on top of each bar
for i, v in enumerate(survival_counts.values):
    axes[0].text(i, v + 5, str(v), ha='center', fontsize=12, fontweight='bold')
# Right plot: Survival rate by gender (grouped bar chart)
survival_by_sex = df.groupby('Sex')['Survived'].value_counts(normalize=True).unstack()
survival_by_sex.plot(kind='bar', ax=axes[1], color=['#FF6B6B', '#4ECDC4'],
                    edgecolor='black', width=0.6)
axes[1].set_title('Survival Rate by Gender', fontsize=14, fontweight='bold')
axes[1].set_ylabel('Proportion')
axes[1].set_xticklabels(['Female', 'Male'], rotation=0)
axes[1].legend(['Did Not Survive', 'Survived'])

```

```
plt.suptitle('Titanic Survival Analysis', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('plot1_survival_bar.png', dpi=150, bbox_inches='tight')
plt.show()
print('Key Insight: 74% of females survived vs only 19% of males!')
```

'Titanic-Dataset.csv' not found. Attempting to download from a public source.

'Titanic-Dataset.csv' downloaded successfully.

/tmp/ipykernel_1815/1351299202.py:23: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chainable assignment. This behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting the value is a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)

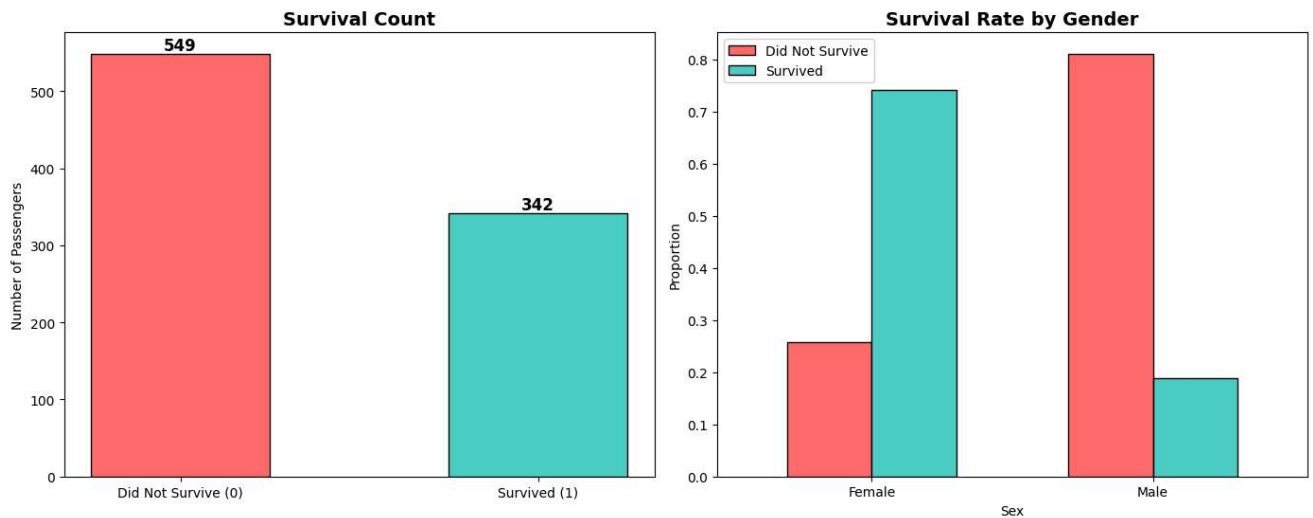
```
df['Age'].fillna(age_medium, inplace=True)
```

/tmp/ipykernel_1815/1351299202.py:25: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chainable assignment. This behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting the value is a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)

```
df['Embarked'].fillna(embarked_mode, inplace=True);
```

Titanic Survival Analysis



Key Insight: 74% of females survived vs only 19% of males!

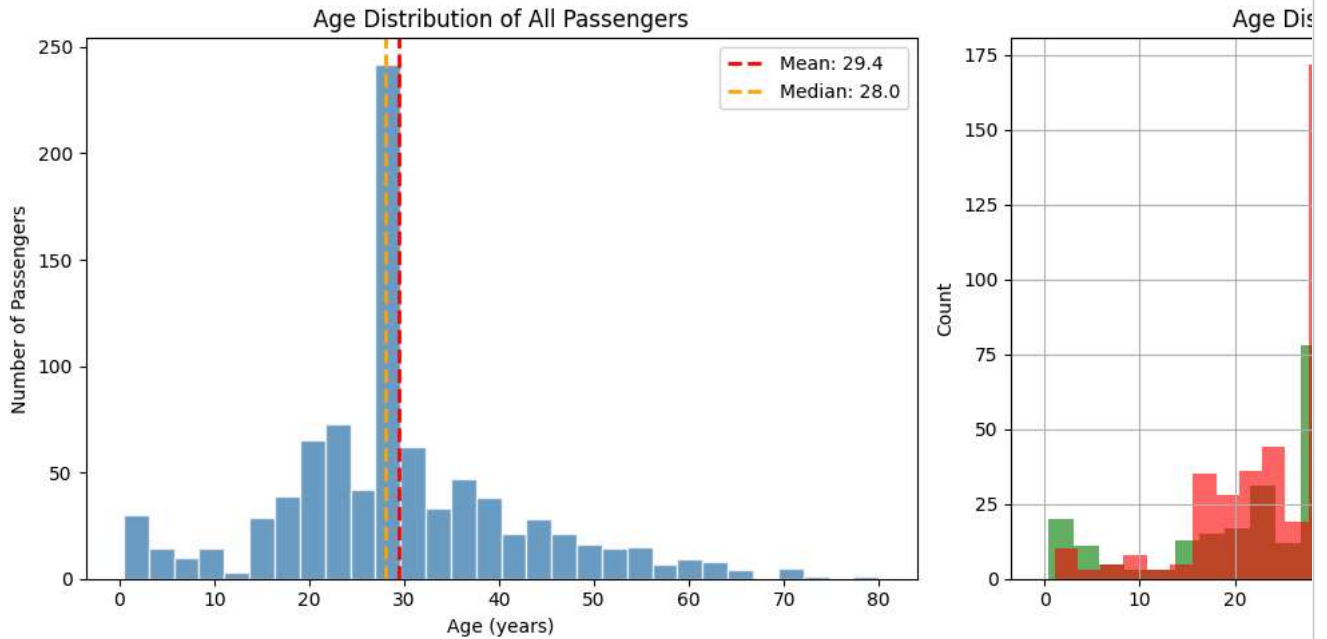
Age distribution histogram.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
# Overall age distribution
axes[0].hist(df['Age'], bins=30, color='steelblue', edgecolor='white', alpha=0.8)
axes[0].axvline(df['Age'].mean(), color='red', linestyle='--', linewidth=2,
label=f"Mean: {df['Age'].mean():.1f}")
axes[0].axvline(df['Age'].median(), color='orange', linestyle='--', linewidth=2,
label=f"Median: {df['Age'].median():.1f}")
axes[0].set_title('Age Distribution of All Passengers')
axes[0].set_xlabel('Age (years)')
axes[0].set_ylabel('Number of Passengers')
axes[0].legend()
# Age distribution by survival (overlaid)
df[df['Survived']==1]['Age'].hist(ax=axes[1], bins=30, alpha=0.6,
color='green', label='Survived')
df[df['Survived']==0]['Age'].hist(ax=axes[1], bins=30, alpha=0.6,
color='red', label='Did Not Survive')
axes[1].set_title('Age Distribution by Survival Status')
axes[1].set_xlabel('Age (years)')
axes[1].set_ylabel('Count')
```

```

axes[1].legend()
plt.tight_layout()
plt.savefig('plot2_age_histogram.png', dpi=150, bbox_inches='tight')
plt.show()

```



Box plots for fare and age.

```

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import os

# Check if the file exists, if not, download it
file_name = 'Titanic-Dataset.csv'
if not os.path.exists(file_name):
    print(f"'{file_name}' not found. Attempting to download from a public source.")
    # Assuming a common public source for the titanic dataset
    !wget -q https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv -O {file_name}
    if os.path.exists(file_name):
        print(f"'{file_name}' downloaded successfully.")
    else:
        raise FileNotFoundError(f"Could not find or download '{file_name}'. Please ensure the file is in the environment or provide a valid URL.")

# Data loading
df = pd.read_csv(file_name)

# Missing value handling
df.drop(columns = ['Cabin'], inplace=True, errors='ignore')
age_medium = df['Age'].median()
df['Age'].fillna(age_medium, inplace=True)
embarked_mode = df['Embarked'].mode()[0]
df['Embarked'].fillna(embarked_mode, inplace=True);

# Feature Engineering
bins = [0, 12, 18, 35, 60, 100]
labels = ['Child', 'Teenager', 'Young Adult', 'Adult', 'Senior']
df['Age_Group'] = pd.cut(df['Age'], bins=bins, labels=labels)
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1
df['IsAlone'] = (df['FamilySize'] == 1).astype(int)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Box plot: Fare by Passenger Class
df.boxplot(column='Fare', by='Pclass', ax=axes[0],
           boxprops=dict(color='navy'),
           medianprops=dict(color='red', linewidth=2))
axes[0].set_title('Fare Distribution by Passenger Class')
axes[0].set_xlabel('Passenger Class')

```

```

axes[0].set_ylabel('Fare (GBP)')
plt.sca(axes[0])
plt.title('Fare Distribution by Passenger Class')
# Seaborn box plot: Age by Survival and Gender
sns.boxplot(data=df, x='Pclass', y='Age', hue='Survived',
palette={0:'salmon', 1:'lightgreen'}, ax=axes[1])
axes[1].set_title('Age Distribution by Class and Survival')
axes[1].set_xlabel('Passenger Class')
axes[1].set_ylabel('Age (years)')
handles, _ = axes[1].get_legend_handles_labels()
axes[1].legend(handles, ['Did Not Survive', 'Survived'])
plt.tight_layout()
plt.savefig('plot3_boxplots.png', dpi=150, bbox_inches='tight')
plt.show()

```

'Titanic-Dataset.csv' not found. Attempting to download from a public source.

'Titanic-Dataset.csv' downloaded successfully.

/tmp/ipykernel_2925/1710879945.py:23: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chainable assignment. This inplace method will never work because the intermediate object on which we are setting the value is a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df

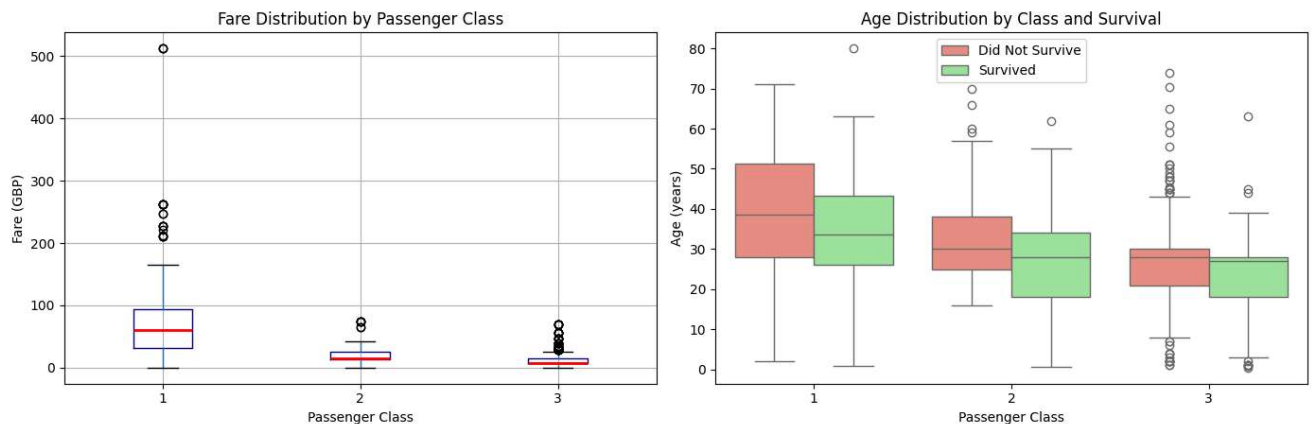
```
df['Age'].fillna(age_medium, inplace=True)
```

/tmp/ipykernel_2925/1710879945.py:25: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chainable assignment. This inplace method will never work because the intermediate object on which we are setting the value is a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df

```
df['Embarked'].fillna(embarked_mode, inplace=True);
```

Boxplot grouped by Pclass



Correlation heatmap.

```

plt.figure(figsize=(10, 7))
# Select only numerical columns for correlation
numerical_cols = ['Survived', 'Pclass', 'Age', 'SibSp', 'Parch', 'Fare',
'FamilySize']
corr_matrix = df[numerical_cols].corr()
# Create the heatmap
sns.heatmap(corr_matrix,
annot=True, # show numbers inside cells
fmt='.2f', # 2 decimal places
cmap='coolwarm', # blue=negative, red=positive
center=0, # white at 0 correlation
square=True, # make cells square
linewidths=0.5, # grid lines between cells
annot_kws={'size': 11})
plt.title('Correlation Heatmap - Titanic Dataset', fontsize=14, fontweight='bold',
pad=20)
plt.tight_layout()

```

```
plt.savefig('plot4_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
print('Key observations from heatmap:')
print('- Fare vs Pclass: -0.55 (1st class passengers paid more)')
print('- Survived vs Fare: +0.26 (higher fare = slightly better survival)')
print('- Survived vs Pclass: -0.34 (lower class = lower survival)')
```

Correlation Heatmap — Titanic Dataset

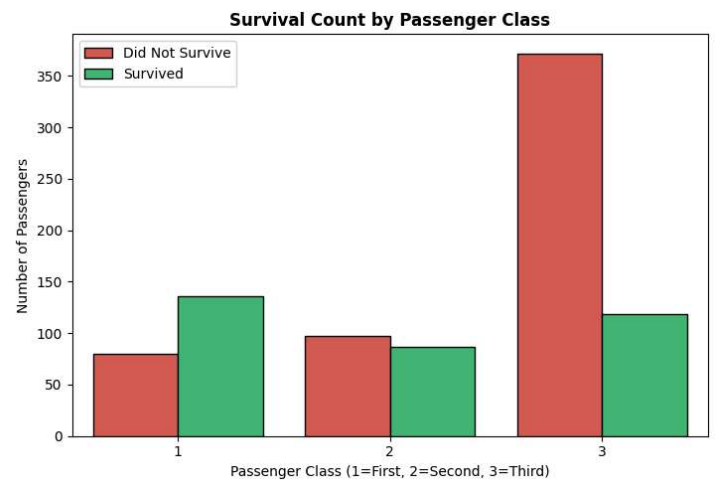
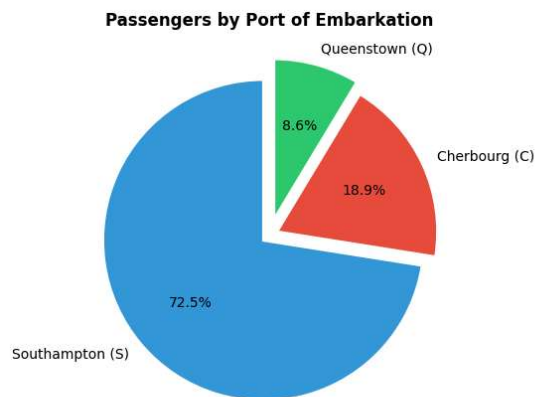


Key observations from heatmap:

- Fare vs Pclass: -0.55 (1st class passengers paid more)
- Survived vs Fare: +0.26 (higher fare = slightly better survival)
- Survived vs Pclass: -0.34 (lower class = lower survival)

Pie chart for embarkation ports.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
# Pie chart: passengers per embarkation port
embarked_counts = df['Embarked'].value_counts()
port_names = {'S': 'Southampton (S)', 'C': 'Cherbourg (C)', 'Q': 'Queenstown (Q)'}
labels = [port_names[p] for p in embarked_counts.index]
explode = (0.05, 0.05, 0.1) # slightly separate the slices
colors = ['#3498DB', '#E74C3C', '#2ECC71']
axes[0].pie(embarked_counts.values, labels=labels, autopct='%1.1f%%',
            explode=explode, colors=colors, startangle=90,
            wedgeprops={'edgecolor': 'white', 'linewidth': 2})
axes[0].set_title('Passengers by Port of Embarkation', fontweight='bold')
# Seaborn countplot: survival by passenger class
sns.countplot(data=df, x='Pclass', hue='Survived',
              palette={0: '#E74C3C', 1: '#2ECC71'}, ax=axes[1],
              edgecolor='black')
axes[1].set_title('Survival Count by Passenger Class', fontweight='bold')
axes[1].set_xlabel('Passenger Class (1=First, 2=Second, 3=Third)')
axes[1].set_ylabel('Number of Passengers')
axes[1].legend(['Did Not Survive', 'Survived'])
plt.tight_layout()
plt.savefig('plot5_pie_countplot.png', dpi=150, bbox_inches='tight')
plt.show()
```



Violin plot - age by class and survival.

```
plt.figure(figsize=(12, 6))
sns.violinplot(data=df, x='Pclass', y='Age', hue='Survived',
               palette={0: '#FF6B6B', 1: '#4ECDC4'},
               split=True, # split violin to compare survived vs not
               inner='quart') # show quartile lines inside violin
plt.title('Age Distribution by Class and Survival Status', fontsize=14,
          fontweight='bold')
plt.xlabel('Passenger Class')
plt.ylabel('Age (years)')
plt.legend(['Did Not Survive', 'Survived'], loc='upper right')
plt.tight_layout()
plt.savefig('plot6_violin.png', dpi=150, bbox_inches='tight')
plt.show()
print('Key Insight: In 3rd class, younger passengers had slightly higher survival rates.')
```

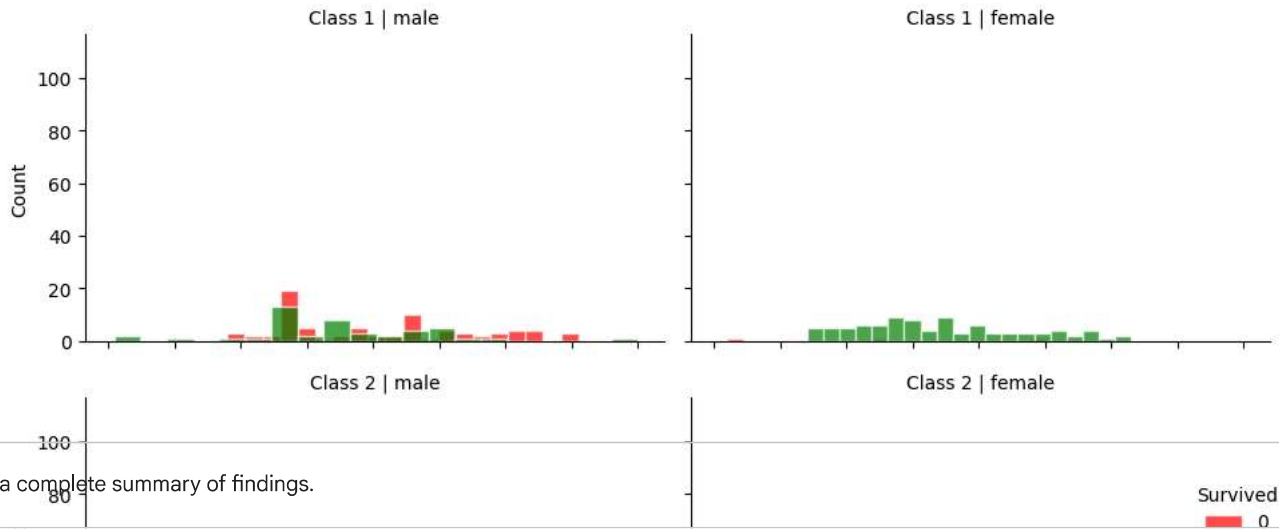


Key Insight: In 3rd class, younger passengers had slightly higher survival rates.

Fact grid - age histogram by class and gender.

```
g = sns.FacetGrid(df, row='Pclass', col='Sex',
hue='Survived', palette={0:'red', 1:'green'},
height=3, aspect=1.5)
g.map(plt.hist, 'Age', bins=20, alpha=0.7, edgecolor='white')
g.add_legend(title='Survived')
g.set_axis_labels('Age', 'Count')
g.set_titles(row_template='Class {row_name}', col_template='{col_name}')
g.figure.suptitle('Age Distribution by Class, Gender & Survival', y=1.02,
fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('plot7_facetgrid.png', dpi=150, bbox_inches='tight')
plt.show()
```

Age Distribution by Class, Gender & Survival



```
print('=' * 60)
print(' TITANIC DATASET - KEY FINDINGS SUMMARY')
print('=' * 60)
total = len(df)
survived = df['Survived'].sum()
print(f'\nTotal Passengers Analysed : {total}')
print(f'Total Survivors : {survived} ({survived/total*100:.1f}%)')
print(f'Total Deaths : {total-survived} ({(total-survived)/total*100:.1f}%)')
print('\n--- SURVIVAL BY GENDER ---')
for sex, rate in df.groupby('Sex')['Survived'].mean().items():
    print(f' {sex.capitalize():<8}: {rate*100:.1f}% survival rate')
print('\n--- SURVIVAL BY CLASS ---')
for cls, rate in df.groupby('Pclass')['Survived'].mean().items():
    label = {1:'First ',2:'Second',3:'Third ')[cls]
    bar = '#' * int(rate * 30)
    print(f' Class {label}: {bar} {rate*100:.1f}%')
print('\n--- AGE STATISTICS ---')
print(f' Average Age: {df["Age"].mean():.1f} years')
print(f' Youngest : {df["Age"].min():.1f} years')
print(f' Oldest : {df["Age"].max():.1f} years')
print('\n--- TOP INSIGHT ---')
print(' Women and children in 1st class had the highest survival rates.')
print(' Men in 3rd class had the lowest survival rates.')
print('=' * 60)
```

```
=====
TITANIC DATASET - KEY FINDINGS SUMMARY
=====
```

```
Total Passengers Analysed : 891
Total Survivors : 342 (38.4%)
Total Deaths : 549 (61.6%)
```